CS683 Programming Assignment II: Stride Pattern Prefetching and Instruction-Conscious L2 Replacement

Sri Charan Raj Kanakala 23B1229 | Dheeraj Thammaneni 23B0997 | Varri Shyam Sri Vardhan 23B0909 | Srivathsa Thotakura 23B1305

***Abstract*—We implement and evaluate, in ChampSim, (i) a Stride Pattern Prefetcher (SPP) at the L1D that separates uniform and non-uniform per-IP stride patterns and throttles its degree by measured accuracy, (ii) SRRIP and an instruction-conscious I-RRIP policy at the L2C, (iii) a dynamic I-RRIP that uses leader sets to decide when instruction prioritisation pays off, and (iv) a prefetch-aware version of dynamic I-RRIP combined with SPP. SPP beats IP-stride on every trace (geometric-mean speedup 1.0091 vs. 1.0023) with a 675-byte budget. I-RRIP cuts L2 instruction MPKI by up to 24%, and the combination of SPP and prefetch-aware dynamic I-RRIP gives the highest geometric-mean speedup of all configurations (1.0152).**

I. Methodology

All results use a single-core ChampSim configuration with a hashed perceptron branch predictor, a 48 KB 12-way L1D (64 sets), a 512 KB 8-way L2C (1024 sets) and a 2 MB 16-way LLC. Every run uses 25M warmup and 25M simulated instructions on the three provided traces. Speedup is $\mathrm{IPC}/\mathrm{IPC}_{\mathrm{baseline}}$, where the baseline is no prefetcher (Task 1, Task 3) or LRU at the L2C (Task 2). L1D coverage is

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{base}} - \text{L1D load misses}_{\text{pf}}}{\text{L1D load misses}_{\text{base}}}.$$

Accuracy is ChampSim's useful prefetches divided by prefetches issued to the lower level. The "GM" bar in each speedup plot is the geometric mean over the three traces.

All three traces have a large code footprint: with LRU, L2 instruction MPKI (10.4, 20.1 and 1.1) is larger than or comparable to L2 data MPKI (7.3, 8.5 and 2.8). This is why data prefetching alone gives modest speedups, and why instruction-conscious replacement matters, especially on trace2.

II. Task 1: Data Prefetching

A. Task 1A: The IP-stride prefetcher

The provided IP-stride prefetcher keeps 24 fully-associative trackers with LRU replacement. Each tracker stores the full IP, the last cache-line address and the last stride. On an access the stride to the previous line is computed; if it equals the stored stride (seen twice in a row), the prefetcher issues $addr + k \cdot stride$ for $k = 1..3$ (fixed degree 3), stopping at the 4 KB page boundary. Lines are filled into the L1D when fewer than half the MSHRs are busy and into the L2C otherwise. It only captures a single constant stride per IP, so a repeating sequence such as $(+1, +2, +2, +2, +3)^n$ never produces two equal consecutive strides for long and is mostly missed, and its degree is fixed regardless of how useful its prefetches are.

B. Task 1B: SPP design

IP table. 64 direct-mapped entries indexed by the low 6 IP bits with a 10-bit partial tag. Each entry stores the low 16 bits of the last line address (enough because strides are limited to one page), the last stride, a 2-bit uniform-stride confidence and a 12-bit stride signature.

Dual pattern classification. The uniform confidence is incremented when the new stride equals the last stride and decremented otherwise. An IP is treated as *uniform* only when the counter is saturated at 3; a pattern like $1, 2, 2, 2, 3$ therefore does not look uniform during its short $2, 2, 2$ run. Uniform IPs are checked first and prefetch $line + k \cdot stride$ for $k = 1..degree$. Otherwise the IP falls back to the non-uniform path.

Non-uniform path. The signature is updated as $sig \leftarrow ((sig \ll 3) \oplus stride) \bmod 2^{12}$, so it summarises the last four strides. A 256-entry pattern table, indexed by $(sig \oplus (sig \gg 8)) \bmod 256$, maps a signature to the next expected stride with a 2-bit confidence. On every access the entry for the *old* signature is trained with the stride that actually happened (increment on a match, otherwise decrement and replace the stride once the confidence reaches 0). To prefetch, SPP looks up the current signature, adds the predicted stride, folds that stride into a speculative signature and looks up again, up to *degree* times. It stops when an entry has confidence below 2 or the next line leaves the page. For the stride sequence $(+1, +2, +2, +2, +3)^n$, once trained every signature has a single successor, so SPP walks the pattern several strides ahead.

Page boundary and fill level. Every candidate is checked against the 4 KB page of the triggering access. Strides larger than 63 lines reset the entry since they cannot stay within the page. Like IP-stride, prefetches fill the L1D when the MSHR is less than half full and the L2C otherwise.

Storage overhead. Table I gives the exact bit-level breakdown: 5397 bits, about 675 bytes.

Accuracy throttling. SPP counts prefetches sent to the lower level and how many of them turned out useful. Every

TABLE I
SPP STORAGE OVERHEAD

Structure / field	Bits	Total
IP table, 64 entries × 48 bits		
valid	1	
IP tag	10	
last line address (low bits)	16	
last stride (−63..63)	7	
uniform confidence	2	
signature	12	
subtotal	48	3072
Pattern table, 256 entries × 9 bits		
next stride	7	
confidence	2	
subtotal	9	2304
Accuracy throttling		
prefetches sent in epoch (0..256)	9	
useful prefetches in epoch (0..256)	9	
current degree (1..4)	3	
subtotal		21
Total		**5397 bits ≈ 675 B**

epoch of 256 sent prefetches it computes the epoch accuracy and sets the degree for the next epoch:

Accuracy	$< 25\%$	25–50%	50–75%	$\geq 75\%$
Degree	1	2	3	4

How the thresholds were chosen. An epoch of 256 prefetches is long enough that one prefetch changes the accuracy by only 0.4%, so the decision is not driven by noise, yet short enough to follow program phases (a few thousand loads). It also keeps both epoch counters at 9 bits. The accuracy range is split into equal quarters because, at degree d and accuracy a, roughly $d \cdot a$ prefetches per trigger are useful and $d(1-a)$ only pollute the L1D and consume bandwidth. Each step up in degree is only allowed once the accuracy band guarantees that most of the extra prefetches will be useful.

Why these degrees. At $\geq 75\%$ the pattern is clearly regular, and looking 4 lines ahead hides more of the memory latency. With accuracy between 25 and 75% we keep a moderate degree of 2–3, which still gives timeliness for the common case. The prefetchers on these traces sit around 45–55% accuracy, where the final degree settled at 3, 2 and 4 on the three traces, showing the degree moving between bands as phases change. Below 25% most prefetches are wasted, so we drop to degree 1 instead of 0: keeping a trickle of prefetches lets the accuracy be measured again and the degree recover in a later epoch. The maximum of 4 matches the number of strides captured in the 12-bit signature; walking further ahead mostly reuses speculative signatures and loses accuracy.

C. Task 1 results

Table II and Figs. 1–4 show the results. SPP outperforms IP-stride on every metric and every trace. Speedups are 1.0069, 1.0052 and 1.0152 (GM 1.0091) against 1.0039, 1.0007 and 1.0022 for IP-stride. Coverage rises from 5.2/0.7/2.5% to 8.0/2.3/11.9%, while accuracy also increases (52.8→55.7%, 38.1→49.5%, 15.9→46.8%). The largest gain is on trace3, where IP-stride's accuracy is only 15.9% because most strides are not constant. SPP's counters show that non-uniform prefetches outnumber uniform ones on all traces (e.g. 594 K vs. 137 K on trace3), so the pattern table is what produces the extra coverage. Absolute speedups remain small because a large part of the stall time comes from L2 instruction misses, which an L1D prefetcher cannot remove.

TABLE II
L1D PREFETCHERS: PERFORMANCE AND PREFETCH QUALITY (25M INSTRUCTIONS)

Trace	Config	IPC	Speedup	L1D MPKI	Acc. (%)	Cov. (%)
trace1	No prefetcher	0.2321	1.0000	10.679	–	–
	IP-stride	0.2330	1.0039	10.119	52.75	5.24
	SPP	0.2337	1.0069	9.824	55.65	8.01
trace2	No prefetcher	0.2499	1.0000	13.535	–	–
	IP-stride	0.2501	1.0007	13.437	38.10	0.72
	SPP	0.2512	1.0052	13.219	49.52	2.34
trace3	No prefetcher	0.3820	1.0000	3.403	–	–
	IP-stride	0.3828	1.0022	3.319	15.88	2.47
	SPP	0.3878	1.0152	2.999	46.77	11.89

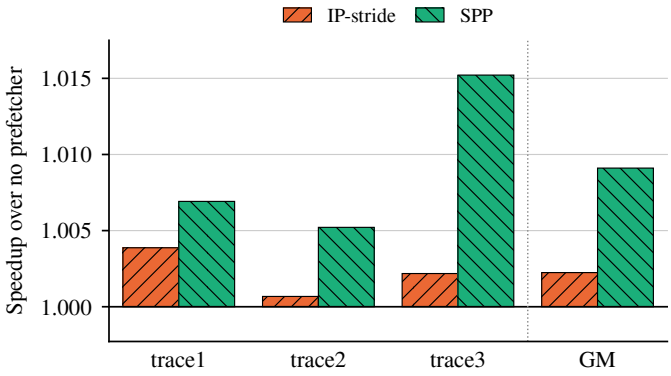


Fig. 1. Task 1 speedup over no prefetching.

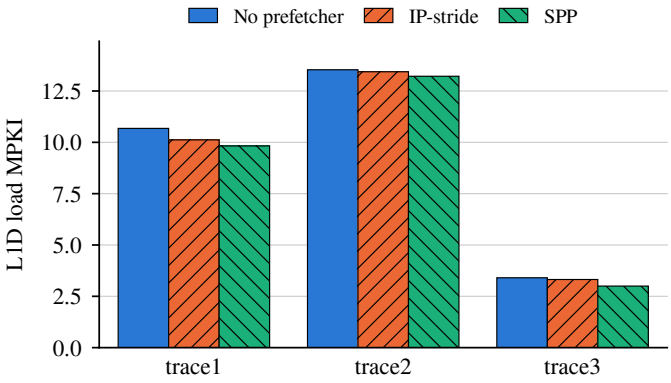


Fig. 2. Task 1 L1D load MPKI.

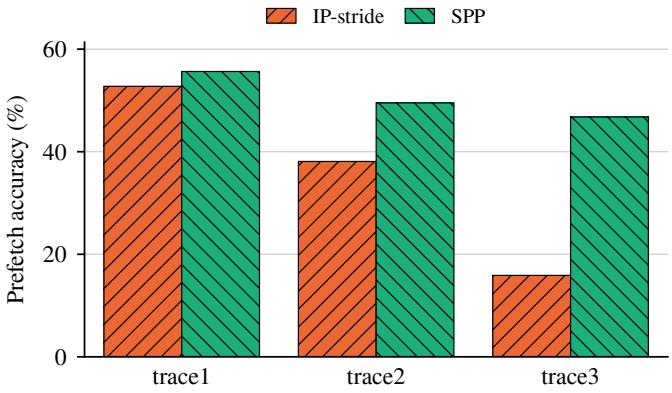


Fig. 3. Task 1 L1D prefetch accuracy.

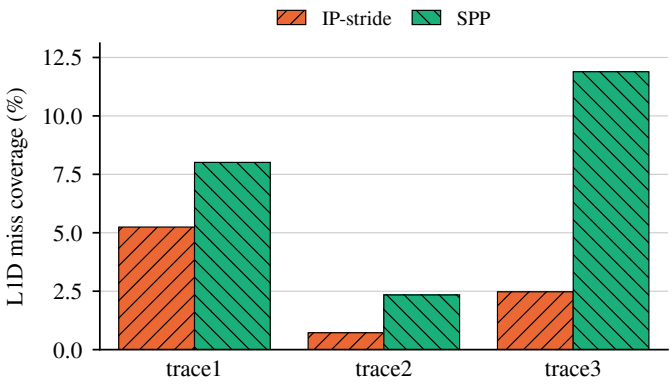


Fig. 4. Task 1 L1D miss coverage.

III. Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

A. Design

The baseline SRRIP uses a 2-bit re-reference prediction value (RRPV) per line: fills are inserted at RRPV 2, hits promote to 0, and the victim is the first line at RRPV 3 (aging the whole set until one exists). I-RRIP adds one bit per line recording whether it holds instructions (from the packet's `instruction` field, passed to `update_replacement_state`) and changes all three parts of the policy:

Insertion. Instruction lines are inserted at RRPV 0, data lines at RRPV 2 as in SRRIP. A code line that missed in the L1I and the L2 will be fetched again soon if the code footprint exceeds the L1I, and an L2 instruction miss stalls the whole frontend, whereas the out-of-order core can overlap many data misses. Inserting code at 0 gives it the longest protection before it can be aged out. Data keeps SRRIP's insertion so that scans and streaming data still leave quickly.

Promotion. An instruction hit goes straight to RRPV 0. A data hit only decrements its RRPV by one. Data therefore has to prove reuse over several hits before it is as protected as code, so a data line touched once or twice cannot push hot code out.

TABLE III
L2C replacement policies, no L1D prefetcher (25M instructions)

Trace	Policy	IPC	Speedup	L2C MPKI	Instr. MPKI	Data MPKI
trace1	LRU	0.2321	1.0000	17.702	10.371	7.331
	SRRIP	0.2320	0.9996	17.608	10.550	7.059
	I-RRIP	0.2319	0.9988	17.773	9.876	7.897
trace2	LRU	0.2499	1.0000	28.644	20.141	8.504
	SRRIP	0.2504	1.0020	27.272	19.325	7.947
	I-RRIP	0.2546	1.0186	24.563	15.299	9.264
trace3	LRU	0.3820	1.0000	3.956	1.120	2.836
	SRRIP	0.3816	0.9990	4.041	1.195	2.845
	I-RRIP	0.3820	1.0001	3.889	0.995	2.894

Eviction. The victim search first looks for a *data* line at RRPV 3 and evicts code at RRPV 3 only when no such data line exists; if neither exists, the set is aged as in SRRIP. Code is never made unevictable: aging still brings it to RRPV 3, so a dead code region is eventually replaced and the policy cannot fill a set with stale instructions.

Changing all three parts is what makes the priority effective. Changing only insertion would let a single data hit promote data to 0, equal to code. Changing only eviction would still insert code at RRPV 2, where it is aged out quickly between frontend re-visits. The cost is 1 bit per line (8 K bits for the 512 KB L2C) on top of SRRIP's 2-bit RRPV.

B. Results

Table III and Figs. 5–8 show the results. I-RRIP reduces L2 instruction MPKI on every trace, from 10.37 to 9.88 (trace1), 20.14 to 15.30 (trace2, -24%) and 1.12 to 1.00 (trace3). As expected, data MPKI rises because data lines now lose to code (e.g. 7.33 to 7.90 on trace1). On trace2, where code is about 70% of the L2 misses, this trade is clearly worth it: total L2C MPKI drops from 28.6 to 24.6 and IPC improves by 1.86%. On trace1 the instruction and data changes cancel out (0.9988), and trace3 has very few instruction misses, so it is unchanged. Plain SRRIP is within $\pm 0.2\%$ of LRU on all traces. The GM speedup is 1.0058 for I-RRIP vs. 1.0002 for SRRIP. The trace1 slowdown motivates the dynamic policy in Task 2B.

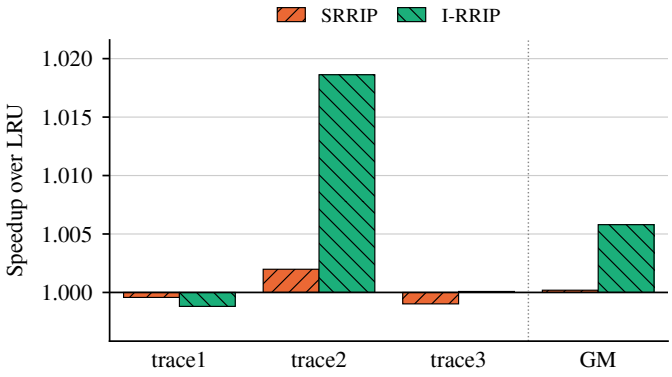


Fig. 5. Task 2A speedup over LRU.

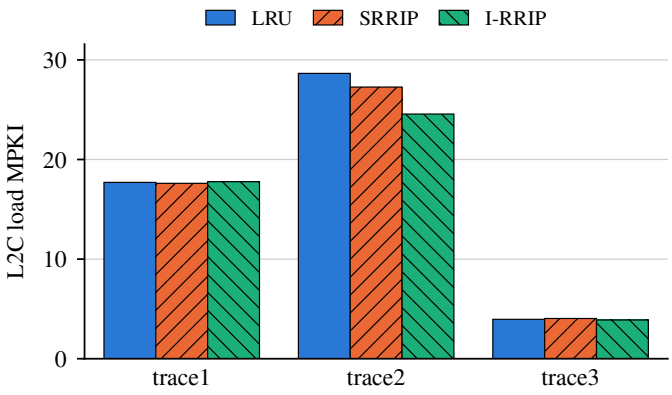


Fig. 6. Task 2A L2C load MPKI.

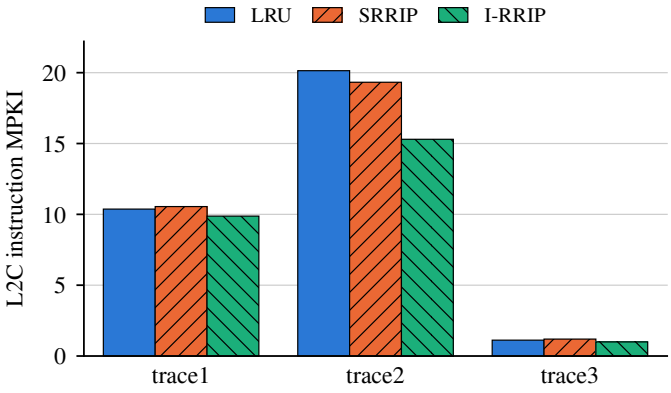


Fig. 7. Task 2A L2C instruction load MPKI.

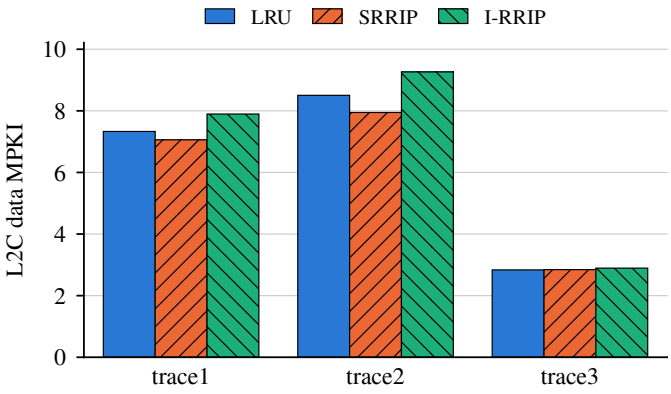


Fig. 8. Task 2A L2C data load MPKI.

IV. Task 2B: Dynamic I-RRIP

A. Design and decision logic

Leader sets. Every 32nd set (sets $0, 32, 64, \ldots$) is a leader, giving 32 leaders out of 1024 sets (3.1%). Leaders always run plain SRRIP, so their statistics describe the workload without being biased by instruction prioritisation. Only 32 leaders are needed because the policy estimates a *ratio*, not which of two policies misses less: with a few thousand accesses per halving interval, 32 sets already give a stable estimate, while keeping 97% of the cache free to use the better policy. Evenly spaced leaders also sample the whole index range, so a hot region of the address space does not bias the estimate.

Decision logic. In the leader sets we count demand accesses and misses, both in total and for instruction lines (writebacks and, for Task 3, prefetches are excluded). Follower sets use I-RRIP when

$$\frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}} > T$$

and SRRIP otherwise. Counting misses as well as accesses weights instruction traffic that actually misses more heavily, which is the traffic that stalls the frontend. All four counters are halved every 4096 leader accesses, so the ratio is an exponentially decaying average and the policy follows phase changes instead of the whole-program mix. The overhead is four counters and the threshold, on top of I-RRIP's per-line bit.

B. Threshold sensitivity

We swept $T \in \{0.05, 0.15, 0.25, 0.35, 0.45\}$ (Figs. 9–10; full data in Table IV). The GM speedups are 1.0050, 1.0050, 1.0047, 1.0045 and 1.0044. A higher threshold makes the followers fall back to SRRIP more often. On trace1 the share of follower fills done in I-RRIP mode falls from 99% at T=0.15 to 0.07% at T=0.45, where the policy becomes identical to SRRIP (instruction MPKI 10.55). On trace2, the most code-heavy trace, the I-RRIP share drops from 92% to 74%, and speedup falls monotonically from 1.0161 to 1.0146. Trace3 is best at $T \leq 0.15$. We use T=0.15: it is tied for the best GM, and it is the largest value that keeps I-RRIP active on trace2 and trace3, which gives some margin against switching too eagerly on data-heavy phases.

C. Comparison

Figs. 11–14 compare dynamic I-RRIP (T=0.15) with LRU, SRRIP and static I-RRIP. It keeps most of I-RRIP's gain on trace2 (1.0161 vs. 1.0186) and matches it elsewhere. It is slightly below static I-RRIP because the 3% leader sets always run SRRIP, and because the ratio measures how much code there is rather than whether prioritising it lowers the miss count. On these three traces code is a large share of the traffic, so the switch mostly selects I-RRIP. The dynamic mechanism matters for data-dominated phases, which it handles by falling back to SRRIP (as the T=0.45 point shows on trace1).

TABLE IV
Dynamic I-RRIP threshold sweep against LRU, SRRIP and I-RRIP (25M instructions)

Trace	Policy	IPC	Speedup	L2C MPKI	Instr. MPKI	Data MPKI
trace1	LRU	0.2321	1.0000	17.702	10.371	7.331
	SRRIP	0.2320	0.9996	17.608	10.550	7.059
	I-RRIP	0.2319	0.9988	17.773	9.876	7.897
	Dyn. I-RRIP (0.05)	0.2319	0.9988	17.763	9.900	7.863
	Dyn. I-RRIP (0.15)	0.2319	0.9988	17.775	9.902	7.873
	Dyn. I-RRIP (0.25)	0.2320	0.9993	17.748	9.884	7.865
	Dyn. I-RRIP (0.35)	0.2320	0.9993	17.745	9.889	7.856
	Dyn. I-RRIP (0.45)	0.2320	0.9996	17.607	10.548	7.059
trace2	LRU	0.2499	1.0000	28.644	20.141	8.504
	SRRIP	0.2504	1.0020	27.272	19.325	7.947
	I-RRIP	0.2546	1.0186	24.563	15.299	9.264
	Dyn. I-RRIP (0.05)	0.2539	1.0161	24.982	15.779	9.203
	Dyn. I-RRIP (0.15)	0.2539	1.0161	24.979	15.777	9.202
	Dyn. I-RRIP (0.25)	0.2538	1.0154	25.118	15.921	9.197
	Dyn. I-RRIP (0.35)	0.2536	1.0149	25.186	16.006	9.181
	Dyn. I-RRIP (0.45)	0.2536	1.0146	25.275	16.128	9.147
trace3	LRU	0.3820	1.0000	3.956	1.120	2.836
	SRRIP	0.3816	0.9990	4.041	1.195	2.845
	I-RRIP	0.3820	1.0001	3.889	0.995	2.894
	Dyn. I-RRIP (0.05)	0.3820	1.0000	3.894	1.003	2.891
	Dyn. I-RRIP (0.15)	0.3820	1.0000	3.891	1.002	2.890
	Dyn. I-RRIP (0.25)	0.3818	0.9994	3.975	1.088	2.887
	Dyn. I-RRIP (0.35)	0.3817	0.9993	4.031	1.143	2.887
	Dyn. I-RRIP (0.45)	0.3816	0.9990	4.049	1.164	2.885

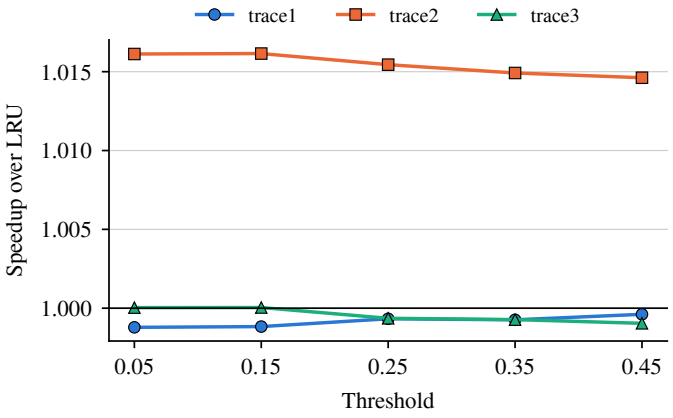


Fig. 9. Task 2B threshold sensitivity: speedup over LRU.

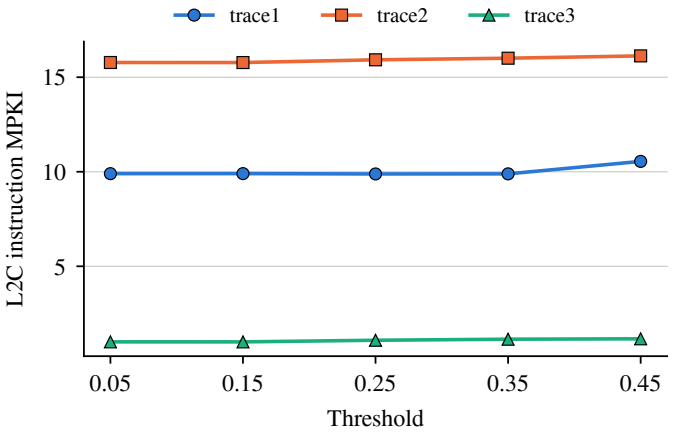


Fig. 10. Task 2B threshold sensitivity: L2C instruction MPKI.

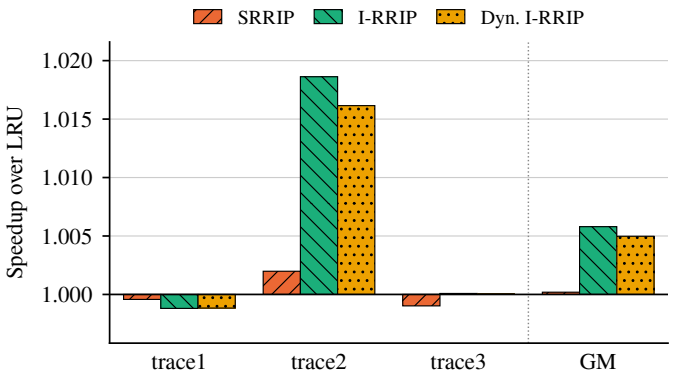


Fig. 11. Task 2B speedup over LRU (T=0.15).

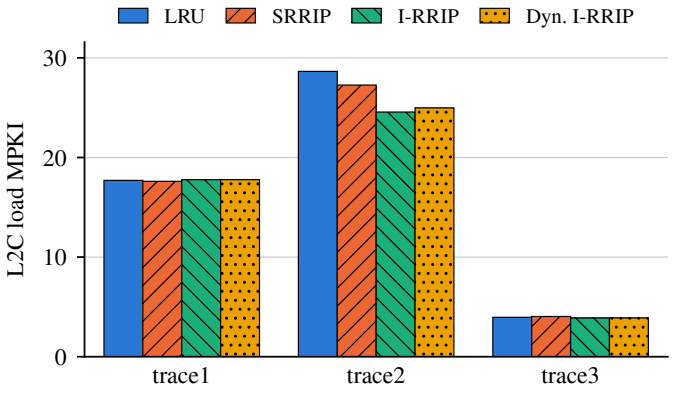


Fig. 12. Task 2B L2C load MPKI.

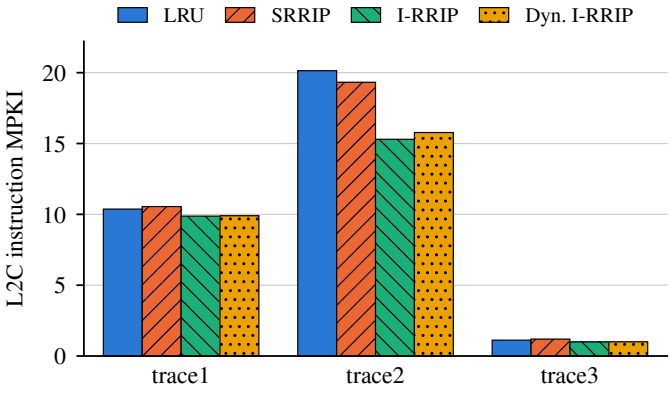


Fig. 13. Task 2B L2C instruction load MPKI.

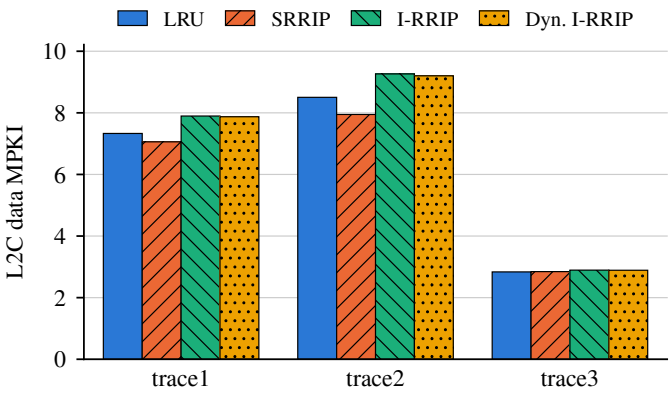


Fig. 14. Task 2B L2C data load MPKI.

V. Task 3: SPP with Prefetch-Aware Dynamic I-RRIP

A. Handling prefetch packets

When SPP fills into the L2C, its prefetches arrive at the replacement policy with type `PREFETCH`. Treating them like demand data has two problems: they distort the leader-set ratio, and the roughly 50% of them that are never used occupy L2 space at the same priority as useful data. The policy handles them as follows:

- **Leader counters ignore prefetches.** Only demand accesses decide whether code is important. Otherwise a burst of data prefetches would lower the instruction ratio and switch off instruction prioritisation exactly when the memory system is busiest.
- **Prefetch fills are inserted at RRPV 3.** With SPP's measured accuracy of 47–56%, about half of the prefetched lines are dead on arrival. Inserting them at the eviction position means a useless prefetch is the next victim in its set and never displaces code or reused data.
- **Prefetch hits do not promote.** A prefetch that hits in the L2 says nothing about demand reuse, so it leaves the RRPV unchanged.
- **Demand hits promote normally.** When the core actually uses a prefetched line, it is promoted like any other data line (to 0 in SRRIP mode, one step in I-RRIP mode), so useful prefetches are kept.

This is in the spirit of prefetch-aware insertion: unproven prefetches get the lowest priority, but a correct prefetch is not penalised once it has been used.

B. Results

Table V and Figs. 15–21 show the results. The combination has the highest GM speedup of all configurations: 1.0152, against 1.0091 for SPP alone and 1.0050 for dynamic I-RRIP alone. It outperforms both components on trace2 (1.0232 vs. 1.0052 and 1.0161) and trace3 (1.0159 vs. 1.0152 and 1.0000). On trace1 it is 0.04% below standalone SPP (1.0065 vs. 1.0069), because dynamic I-RRIP alone is slightly negative on trace1 (Task 2A). The gains add up because the two mechanisms remove different misses: SPP removes L1D data misses, and I-RRIP removes L2 instruction misses. The combination also has the lowest L2C load MPKI on every trace (16.99, 24.63 and 3.30). L2 data MPKI with SPP stays well below the no-prefetch dynamic I-RRIP configuration (e.g. 2.31 vs. 2.89 on trace3), showing that the low-priority insertion keeps prefetches from polluting the L2. SPP's accuracy and coverage are essentially unchanged by the L2 policy (Figs. 17, 18), as expected for an L1D prefetcher.

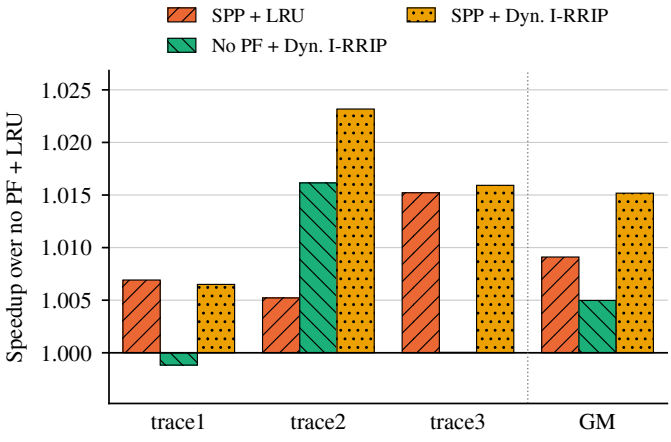


Fig. 15. Task 3 speedup over no prefetching with LRU.

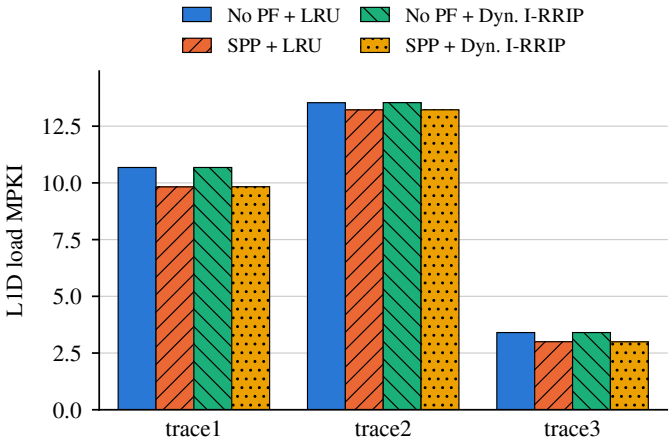


Fig. 16. Task 3 L1D load MPKI.

TABLE V
SPP COMBINED WITH PREFETCH-AWARE DYNAMIC I-RRIP (25M INSTRUCTIONS)

Trace	Config	IPC	Speedup	Acc. (%)	Cov. (%)	L1D	L2C	L2 instr.	L2 data
trace1	No PF + LRU	0.2321	1.0000	–	–	10.679	17.702	10.371	7.331
	SPP + LRU	0.2337	1.0069	55.65	8.01	9.824	17.024	10.411	6.613
	No PF + Dyn. I-RRIP	0.2319	0.9988	–	–	10.680	17.775	9.902	7.873
	SPP + Dyn. I-RRIP	0.2336	1.0065	55.60	7.95	9.830	16.986	9.890	7.096
trace2	No PF + LRU	0.2499	1.0000	–	–	13.535	28.644	20.141	8.504
	SPP + LRU	0.2512	1.0052	49.52	2.34	13.219	28.680	20.449	8.231
	No PF + Dyn. I-RRIP	0.2539	1.0161	–	–	13.538	24.979	15.777	9.202
	SPP + Dyn. I-RRIP	0.2557	1.0232	49.96	2.32	13.221	24.628	15.684	8.944
trace3	No PF + LRU	0.3820	1.0000	–	–	3.403	3.956	1.120	2.836
	SPP + LRU	0.3878	1.0152	46.77	11.89	2.999	3.402	1.130	2.271
	No PF + Dyn. I-RRIP	0.3820	1.0000	–	–	3.403	3.891	1.002	2.890
	SPP + Dyn. I-RRIP	0.3881	1.0159	47.03	12.00	2.995	3.304	0.995	2.309

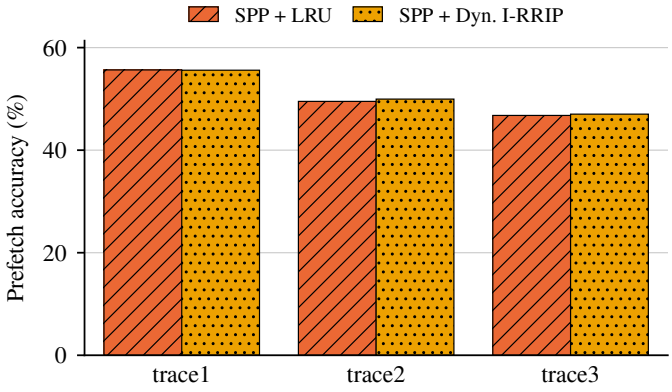


Fig. 17. Task 3 L1D prefetch accuracy.

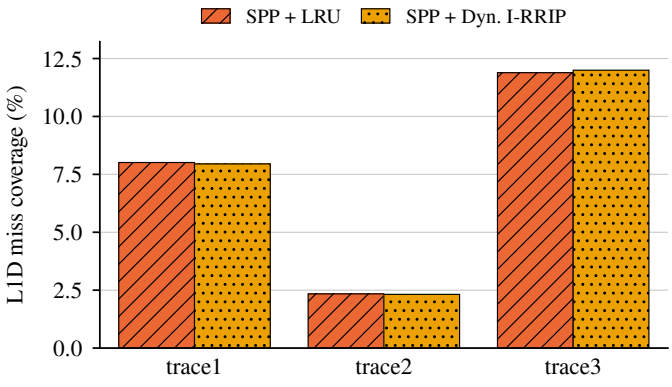


Fig. 18. Task 3 L1D miss coverage.

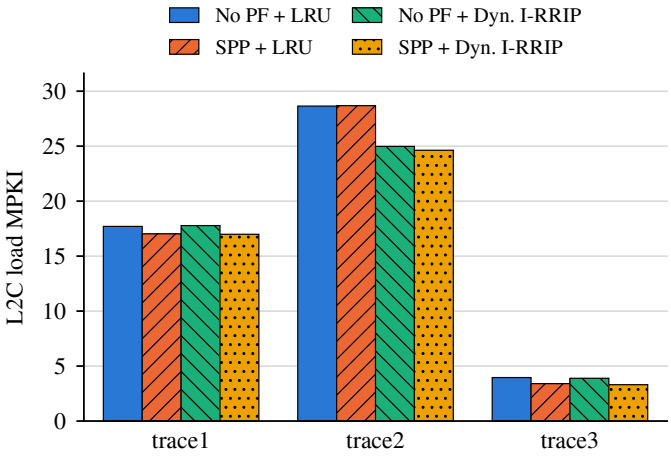


Fig. 19. Task 3 L2C load MPKI.

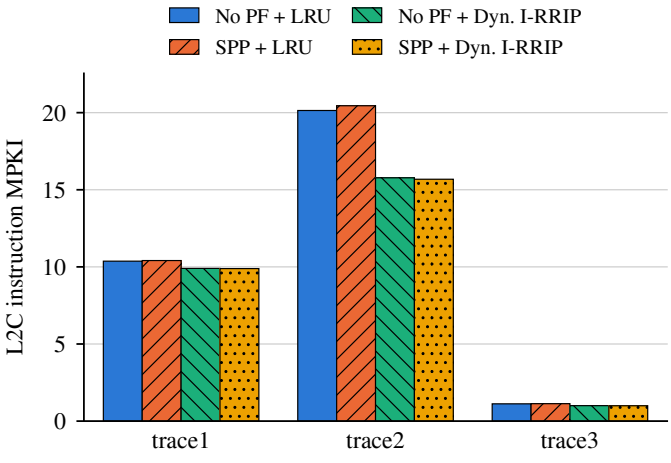


Fig. 20. Task 3 L2C instruction load MPKI.

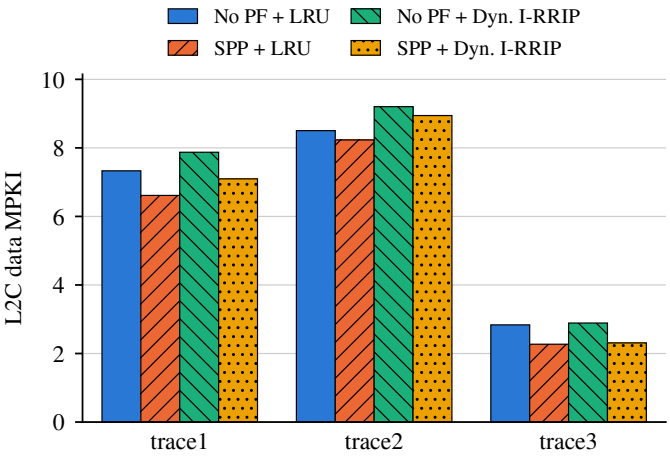


Fig. 21. Task 3 L2C data load MPKI.

VI. Conclusion

SPP's dual uniform / signature-based classification and accuracy throttling give consistently higher accuracy, coverage and speedup than IP-stride for 675 B of state. At the L2C, prioritising instructions in insertion, promotion and eviction cuts instruction MPKI by up to 24%. The leader-set based dynamic policy keeps that benefit while falling back to SRRIP when code is a small share of the traffic. Making the policy prefetch-aware lets the two mechanisms work together, giving the best overall speedup (GM 1.0152) and the lowest L2C MPKI on all traces.